

```

#include <Wire.h>
#include <Adafruit_TCS34725.h>
#include <LiquidCrystal_I2C.h>
#include <ESP32Servo.h>

// Pin definitions
#define MOTOR_PIN 23
#define SERVO_PIN 18
#define RELAY1_PIN 13 // Bin 1
#define RELAY2_PIN 12 // Bin 2
#define RELAY3_PIN 14 // Bin 3
#define IR1_PIN 19    // First IR (object detection, active LOW)
#define IR2_PIN 5     // Second IR (bin full, active LOW)
#define BUZZER_PIN 4

// Color matching threshold (tune based on testing)
#define MAX_COLOR_DIST 0.05 // Adjust after testing known/unknown colors

// Initialize components
Adafruit_TCS34725 tcs = Adafruit_TCS34725(TCS34725_INTEGRATIONTIME_50MS,
TCS34725_GAIN_4X);
LiquidCrystal_I2C lcd(0x27, 16, 2);
Servo servo;

// Color storage (RGB for 3 colors)
struct Color {
    float r, g, b;
};
Color calibratedColors[3];
int currentColorIndex = 0; // For calibration
int sortedCounts[3] = {0, 0, 0}; // Separate counts for each color

// System states
enum State { CALIBRATING, RUNNING, UNKNOWN_COLOR, BIN_FULL };
State systemState = CALIBRATING;

// Flag to wait for object to pass after detection
bool waitingForObjectPass = false;
int lastDetectedColor = -1; // To remember color for increment after pass

void setup() {
    // Initialize pins
    pinMode(MOTOR_PIN, OUTPUT);
    pinMode(RELAY1_PIN, OUTPUT);
    pinMode(RELAY2_PIN, OUTPUT);

```

```

pinMode(RELAY3_PIN, OUTPUT);
pinMode(BUZZER_PIN, OUTPUT);
pinMode(IR1_PIN, INPUT);
pinMode(IR2_PIN, INPUT);

digitalWrite(MOTOR_PIN, HIGH); // Off (inverted logic)
digitalWrite(RELAY1_PIN, LOW);
digitalWrite(RELAY2_PIN, LOW);
digitalWrite(RELAY3_PIN, LOW);
digitalWrite(BUZZER_PIN, LOW);

// Initialize Serial
Serial.begin(115200);
Serial.println("System Starting...");

// Initialize I2C components
if (!tcs.begin()) {
    Serial.println("TCS34725 not found!");
    while (1); // Halt if sensor fails
}
lcd.init();
lcd.backlight();
lcd.setCursor(0, 0);
lcd.print("Place 1st color");

// Initialize servo
servo.attach(SERVO_PIN);
}

void loop() {
    switch (systemState) {
        case CALIBRATING:
            handleCalibration();
            break;
        case RUNNING:
            handleRunning();
            break;
        case UNKNOWN_COLOR:
            handleUnknownColor();
            break;
        case BIN_FULL:
            handleBinFull();
            break;
    }
}

```

```

// Read and average 20 RGB readings with normalization
Color readAverageColor() {
    float rSum = 0, gSum = 0, bSum = 0, cSum = 0;
    uint16_t r, g, b, c;

    for (int i = 0; i < 20; i++) { // Increased to 20 readings
        tcs.getRawData(&r, &g, &b, &c);
        rSum += r;
        gSum += g;
        bSum += b;
        cSum += c;
        delay(100); // Delay between readings for stability
    }

    Color avg;
    avg.r = rSum / 20.0;
    avg.g = gSum / 20.0;
    avg.b = bSum / 20.0;
    float avgC = cSum / 20.0;

    // Normalize RGB by clear channel to reduce lighting effects
    if (avgC > 0) {
        avg.r /= avgC;
        avg.g /= avgC;
        avg.b /= avgC;
    }

    Serial.print("Normalized RGB: ");
    Serial.print(avg.r); Serial.print(", ");
    Serial.print(avg.g); Serial.print(", ");
    Serial.println(avg.b);

    return avg;
}

// Calculate Euclidean distance between two colors
float colorDistance(Color c1, Color c2) {
    return sqrt(pow(c1.r - c2.r, 2) + pow(c1.g - c2.g, 2) + pow(c1.b - c2.b, 2));
}

// Find closest calibrated color; return -1 if unknown (too distant)
int findClosestColor(Color detected) {
    float minDist = colorDistance(detected, calibratedColors[0]);
    int closest = 0;

```

```

    for (int i = 1; i < 3; i++) {
        float dist = colorDistance(detected, calibratedColors[i]);
        if (dist < minDist) {
            minDist = dist;
            closest = i;
        }
    }

    Serial.print("Min distance: "); Serial.println(minDist);
    if (minDist > MAX_COLOR_DIST) {
        Serial.println("Color unknown (distance too large)");
        return -1;
    }

    Serial.print("Closest color index: "); Serial.println(closest);
    return closest;
}

// Update LCD with running status and counts
void updateLCDCounts() {
    lcd.clear();
    lcd.print("Running...");
    lcd.setCursor(0, 1);
    lcd.print("C1:");
    lcd.print(sortedCounts[0]);
    lcd.print(" C2:");
    lcd.print(sortedCounts[1]);
    lcd.print(" C3:");
    lcd.print(sortedCounts[2]);
}

void handleCalibration() {
    static bool waitingForObject = true;

    if (waitingForObject && digitalRead(IR1_PIN) == LOW) { // Active LOW: object
detected
        Serial.println("Object detected, scanning...");
        delay(50); // Debounce IR
        Color avgColor = readAverageColor();
        calibratedColors[currentColorIndex] = avgColor;
        lcd.clear();
        lcd.print("Scan complete");
        lcd.setCursor(0, 1);
        lcd.print("Remove object");
    }
}

```

```

        waitingForObject = false;
    }
    else if (!waitingForObject && digitalRead(IR1_PIN) == HIGH) { // Active LOW:
object removed
        Serial.println("Object removed");
        delay(50); // Debounce IR
        currentColorIndex++;

        if (currentColorIndex < 3) {
            lcd.clear();
            lcd.print("Place ");
            lcd.print(currentColorIndex + 1);
            lcd.print("th color");
            waitingForObject = true;
        }
        else {
            lcd.clear();
            lcd.print("Calibration done");
            lcd.setCursor(0, 1);
            lcd.print("Starting...");
            delay(2000); // Show message briefly
            digitalWrite(MOTOR_PIN, LOW); // On (inverted logic)
            systemState = RUNNING;
            updateLCDCounts();
            Serial.println("Calibration complete, starting conveyor");
        }
    }
}

void handleRunning() {
    // Check for object pass (rising edge)
    if (waitingForObjectPass && digitalRead(IR1_PIN) == HIGH) { // Object passed
        delay(50); // Debounce IR
        waitingForObjectPass = false;
        if (lastDetectedColor != -1) { // Valid color was detected
            sortedCounts[lastDetectedColor]++;
            updateLCDCounts();
            Serial.println("Object passed, count incremented");
        }
        lastDetectedColor = -1; // Reset
    }

    // Check for new object arrival (falling edge)
    if (!waitingForObjectPass && digitalRead(IR1_PIN) == LOW) { // Active LOW:
object detected

```

```

Serial.println("Object detected on belt");
delay(50); // Debounce IR
Color detectedColor = readAverageColor();
int closestColor = findClosestColor(detectedColor);

if (closestColor == -1) {
    // Unknown color
    digitalWrite(MOTOR_PIN, HIGH); // Off (inverted)
    digitalWrite(BUZZER_PIN, HIGH);
    lcd.clear();
    lcd.print("Unknown color!");
    lcd.setCursor(0, 1);
    lcd.print("Remove object");
    systemState = UNKNOWN_COLOR;
    Serial.println("Unknown color detected");
    return;
}

// Show detected color on LCD
lcd.clear();
lcd.print("Detected C");
lcd.print(closestColor + 1);
delay(1000); // Show briefly

// Restore running display
updateLCDCounts();

Serial.println("Color recognized, moving servo");
int angle;
int relayPin;
if (closestColor == 0) {
    angle = 67;
    relayPin = RELAY1_PIN;
}
else if (closestColor == 1) {
    angle = 98;
    relayPin = RELAY2_PIN;
}
else {
    angle = 130;
    relayPin = RELAY3_PIN;
}

servo.write(angle);
delay(500); // Allow servo to settle

```

```

// Check if bin is full
delay(100); // Wait for object to reach bin
if (digitalRead(IR2_PIN) == LOW) { // Active LOW: bin full
    Serial.println("Bin full detected");
    digitalWrite(MOTOR_PIN, HIGH); // Off (inverted)
    digitalWrite(BUZZER_PIN, HIGH);
    digitalWrite(relayPin, HIGH);
    lcd.clear();
    lcd.print("Bin ");
    lcd.print(closestColor + 1);
    lcd.print(" full!");
    lcd.setCursor(0, 1);
    lcd.print("Empty it");
    systemState = BIN_FULL;
}
else { // Bin not full, wait for object to pass
    waitingForObjectPass = true;
    lastDetectedColor = closestColor;
    Serial.println("Waiting for object to pass");
}
}
// If no object, keep running
}

void handleUnknownColor() {
    if (digitalRead(IR1_PIN) == HIGH) { // Active LOW: object removed
        Serial.println("Unknown object removed");
        delay(50); // Debounce IR
        digitalWrite(MOTOR_PIN, LOW); // On (inverted)
        digitalWrite(BUZZER_PIN, LOW);
        updateLCDCounts();
        systemState = RUNNING;
    }
}

void handleBinFull() {
    static unsigned long emptyTime = 0;
    static bool waitingForEmpty = true;

    if (waitingForEmpty && digitalRead(IR2_PIN) == HIGH) { // Active LOW: bin
emptied
        Serial.println("Bin emptied");
        delay(50); // Debounce IR
        emptyTime = millis();
    }
}

```

```
    waitingForEmpty = false;
}
else if (!waitingForEmpty && (millis() - emptyTime >= 5000)) {
    digitalWrite(BUZZER_PIN, LOW);
    digitalWrite(RELAY1_PIN, LOW);
    digitalWrite(RELAY2_PIN, LOW);
    digitalWrite(RELAY3_PIN, LOW);
    digitalWrite(MOTOR_PIN, LOW); // On (inverted)
    updateLCDCounts();
    systemState = RUNNING;
    waitingForEmpty = true;
    Serial.println("Resuming after bin empty");
}
}
```